



Alchemy Guild: The 3-3-1 Waterfall Loadout & Degen Engine

Target Network: Arbitrum Sepolia (421614)

Deadline Context: 5-Day Hackathon Sprint (DoraHacks). Prioritize execution speed, gas efficiency ($O(1)$ where possible), and strict adherence to the existing frontend UI.



STRONGLY ENFORCED CONSTRAINTS

1. **DO NOT MODIFY** ElementNFT.sol or Alchemist.sol. The minting (0.002 ETH) and crafting (Burn 3 to upgrade) mechanics remain 100% untouched.
2. **DO NOT REQUIRE UI CHANGES.** The frontend UI displays Tier Yield Multipliers (Lead 100%, Silver 135%, Gold 175%). The smart contract math must support these fixed weights dynamically without requiring UI rewrites.



1. Architecture Upgrade: GuildDistributor.sol

We are upgrading GuildDistributor.sol from a flat pro-rata distributor into a **Weighted Waterfall** staking contract combined with an RPG "**Equip Loadout**" system.

Players stake their Element NFTs here to earn real USDC yield harvested from our DeFi engine.

The "Equip Loadout" Limits (Anti-Whale)

To prevent a single heavily funded wallet from crafting unlimited Tier 3 elements and draining the protocol, we are enforcing strict equip limits per address. A single smart account can only stake a maximum of **7 Elements** at any given time.

- **Tier I (Lead):** Max 3 equipped (Base Weight: 100)
- **Tier II (Silver):** Max 3 equipped (Base Weight: 135)
- **Tier III (Gold):** Max 1 equipped (Base Weight: 175)

Implementation Directive: Use uint8 mapping counters in the staking function (e.g., `mapping(address => uint8) public stakedGoldCount;`). Do not use unbounded for-loops. Revert the transaction if a user attempts to stake beyond their tier limit. Decrement counters upon withdrawal.



2. The Weighted Waterfall Logic (Triple Accumulator)

When USDC fees are injected into the Distributor via `notifyRewardAmount()`, they are split into three distinct pools.

- **Pool 1 (The Base Stream - e.g., 50%):** Accessible by Tier I, II, and III.

- **Pool 2 (The Adept Stream - e.g., 30%):** Accessible by Tier II and III ONLY.
- **Pool 3 (The Master Stream - e.g., 20%):** Accessible by Tier III ONLY.

Smart Contract Math (Synthetix-Style)

To execute this, GuildDistributor.sol must track three separate global weights and three separate rewardPerWeight accumulators.

1. **Global Trackers:** Update totalWeightT1, totalWeightT2, and totalWeightT3 whenever a user stakes or withdraws.
2. **The Injection (notifyRewardAmount):**
 - Split the incoming USDC based on the defined ratio (50/30/20).
 - $\text{rewardPerWeightPool1} += (\text{Pool1Amount} * 1e18) / (\text{totalWeightT1} + \text{totalWeightT2} + \text{totalWeightT3})$
 - $\text{rewardPerWeightPool2} += (\text{Pool2Amount} * 1e18) / (\text{totalWeightT2} + \text{totalWeightT3})$
 - $\text{rewardPerWeightPool3} += (\text{Pool3Amount} * 1e18) / \text{totalWeightT3}$
3. **The Claim Math (earned):**
 - Loop through a user's staked tokenIds.
 - Look up the Tier via ElementNFT.getElementTier(tokenId).
 - Apply the appropriate Weight (100, 135, or 175).
 - **Tier 1 Earns:** $\text{Weight} * (\text{Pool1Reward} - \text{UserPool1Paid})$
 - **Tier 2 Earns:** $\text{Weight} * ((\text{Pool1} + \text{Pool2}) - (\text{UserPaid}))$
 - **Tier 3 Earns:** $\text{Weight} * ((\text{Pool1} + \text{Pool2} + \text{Pool3}) - (\text{UserPaid}))$



3. The Degen Engine: GUILD/WETH Pool Creation

To power this Flywheel, the protocol requires a high-risk Uniswap V3 pool to generate fees. Since GUILD is a custom token, you must initialize this pool on Arbitrum Sepolia in your Foundry deployment script.

Required Addresses (Arbitrum Sepolia):

- NFPM: 0xC36442b4a4522E871399CD717aBDD847Ab11FE88
- GUILD: 0x39514660f913E651E098c710b03943bA5F451535
- WETH: 0x980B62Da83eFf3D4576C647993b0c1D7faf17c73

Deployment Script Directive (Deploy.s.sol):

1. Sort GUILD and WETH so token0 < token1.
2. Use a **1% Fee Tier (10000)** to maximize degen fee generation.
3. Calculate the initialSqrtPriceX96 for a 1:1 starting ratio (e.g., 79228162514264337593543950336 if decimals match).
4. Execute INonfungiblePositionManager(NFPM).createAndInitializePoolIfNecessary(...).



4. Hackathon Emergency Fallback (The Cheat Code)

Goal: Ensure a flawless live demo even if DEX routing integration stalls.

If wiring up YieldVault.sol to harvest from the NFPM, swap via the Uniswap Router, and push to GuildDistributor becomes too complex for the deadline:

1. Prioritize perfectly executing the GuildDistributor.sol Staking UI and Waterfall math.
2. Restrict notifyRewardAmount(uint256 amount) to onlyOwner.
3. During the presentation, we will bypass YieldVault.sol and manually mint/transfer testnet USDC directly to the GuildDistributor as the owner to simulate the Flywheel injecting yield in real-time.

END OF SPEC